

## บทที่ 3

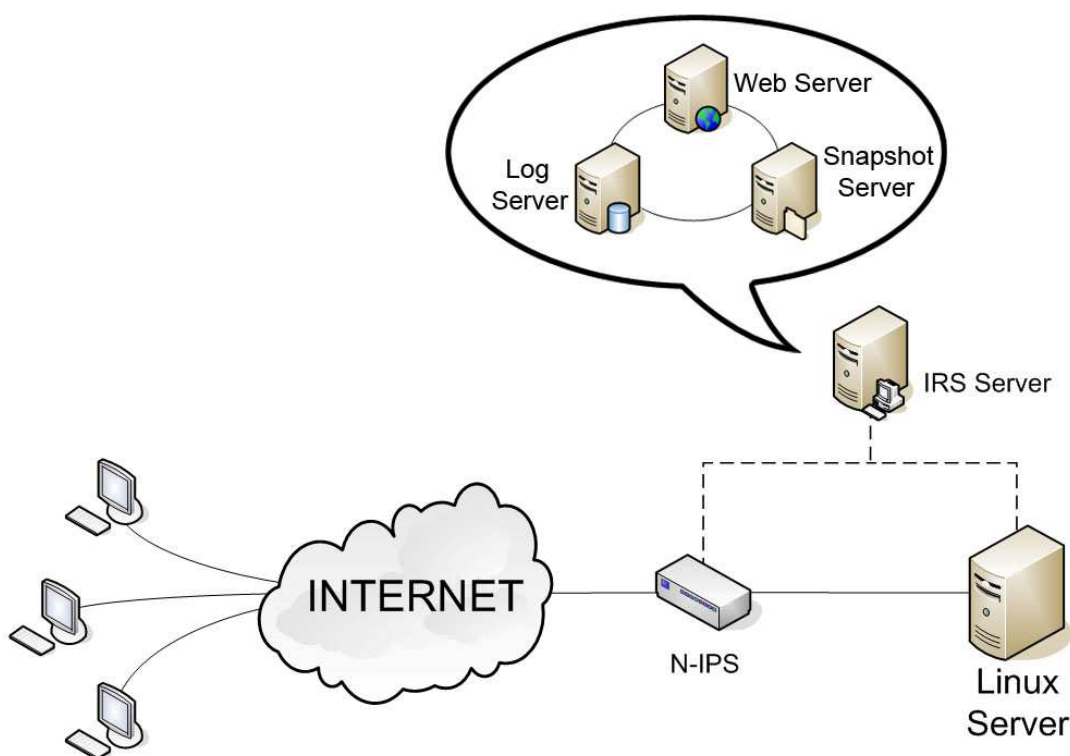
### การออกแบบและพัฒนา

#### 3.1 บทนำ

ในส่วนของการออกแบบและพัฒนานั้นจะต้องมีการพิจารณาถึงองค์ประกอบต่างๆ ที่จะนำมารวมและสร้างเป็นระบบขึ้น เพื่อให้ทำงานได้ตามเป้าหมายที่วางไว้ โดยในส่วนต่างๆ นั้นได้มีการพิจารณาอย่างเหมาะสม

นอกจากนั้นสิ่งสำคัญที่จะต้องนำมาพิจารณาคือ ความปลอดภัยของระบบ ทั้งนี้ระบบที่สร้างขึ้นจะต้องมีความปลอดภัยในตัวเองในระดับหนึ่ง ดังนั้นการออกแบบจึงต้องมีความรัดกุมและรอบคอบมากที่สุด โดยรายละเอียดของการออกแบบในแต่ละส่วนนั้นจะอธิบายในหัวข้อถัดไปซึ่งแยกเป็น การออกแบบฮาร์ดแวร์ และการออกแบบซอฟต์แวร์

#### 3.2 การออกแบบฮาร์ดแวร์



รูปที่ 3.1 แผนผังการออกแบบ

จากรูปภาพแสดงให้เห็นส่วนต่างๆ ของระบบประกอบไปด้วย

- N-IPS ทำหน้าที่ตรวจจับการบุกรุกและทำหน้าที่ในการตอบสนองเบื้องต้น

- Linux Server เป็น Server ที่ให้บริการ
- IRS Server จะแบ่งได้ออกเป็น 3 ส่วน คือ
  1. Log Server
  2. Snapshot Server
  3. Web Server ที่ใช้เป็น admin control

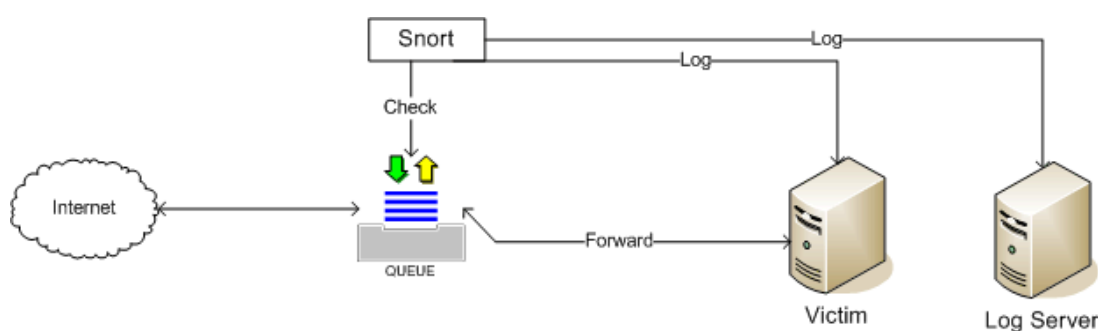
### 3.3 การออกแบบซอฟต์แวร์

ในการออกแบบซอฟต์แวร์จะแบ่งออกเป็น 3 ส่วนดังนี้คือ

1. ระบบตรวจจับผู้บุกรุกทางเครือข่าย
2. ระบบตรวจจับและป้องกันผู้บุกรุกทางคอมพิวเตอร์
3. การกู้คืนระบบ

#### 3.3.1 ระบบป้องกันผู้บุกรุกทางเครือข่าย

จะเป็นการป้องกันด่านแรกเพื่อคอยตรวจจับผู้บุกรุกที่พยายามบุกรุกผ่านทางเครือข่ายซึ่งจะสามารถตรวจจับและป้องกันโดยในที่นี้เราใช้ tools ตัวหนึ่งที่ชื่อ snort-inline นำมาพัฒนาซึ่งตัว snort-inline นั้นจะมีการตรวจจับการบุกรุกเป็นแบบ pattern matching หรือก็คือจะเป็นการตรวจดูรูปแบบในการบุกรุกที่ได้จดจำไว้ซึ่งถ้าเป็นการบุกรุกรูปแบบใหม่ๆที่ระบบไม่รู้จักรก็จะไม่สามารถป้องกันได้แต่ตัวระบบของเราจะเก็บข้อมูลที่ผ่านทางเครือข่ายทั้งหมดเก็บไว้เพื่อใช้ในการวิเคราะห์หากเกิดปัญหาหรือคูช่องโหว่ของระบบที่ผู้บุกรุกพยายามที่จะบุกรุกเข้ามาซึ่งข้อมูลต่างๆเหล่านี้จะเก็บลงบนฐานข้อมูล



รูปที่ 3.2 การทำงานของ snort

#### 3.3.2 ระบบตรวจจับผู้บุกรุกทางคอมพิวเตอร์

จะแบ่งออกเป็น 3 ส่วนคือ

1. ส่วนของ Anti Exploit
2. ส่วนของเคอร์เนล โมดูล

### 3. ส่วนของการตรวจสอบความสมบูรณ์ของไฟล์

โดยการทำงานนั้นจะทำงานร่วมกันกล่าวคือถ้าผู้บุกรุกสามารถบุกรุกผ่านทางเครือข่ายเข้ามาในระบบของเราแล้วนั้นสิ่งแรกที่ผู้บุกรุกนั้นจะพยายามทำคือการยกระดับสิทธิ์ของตนเองให้เท่าเทียมกับผู้ดูแลระบบซึ่งสิ่งที่ผู้บุกรุกจะนิยมทำก็คือการทำ exploit ด้วยการทำ บัฟเฟอร์โอเวอร์โฟลว์ ซึ่งทางผู้จัดทำได้สร้างตัว Anti-Exploit ขึ้นมาเพื่อคอยดักเมื่อมีการทำ exploit เกิดขึ้นระบบจะทำการ kill process ที่ทำการ run ทิ้งไป และเก็บข้อมูลลงฐานข้อมูลโดยตัวการทำ บัฟเฟอร์โอเวอร์โฟลว์ ที่นิยมใช้กันก็คือการเรียกใช้ผ่าน function ของภาษา C ซึ่งตัว Anti-Exploit ที่ได้จัดทำขึ้นนั้นได้ใช้เทคนิคของตัว LD\_Preload คือเมื่อมีการเรียกใช้ function ภาษา C จะต้องมาเรียกผ่าน ไบบรารี ที่ได้จัดทำขึ้นเพื่อดูว่า function ที่เรียกนั้นได้ทำ บัฟเฟอร์โอเวอร์โฟลว์ ระบบหรือเปล่าถ้าทำก็จะ Kill process ทิ้งไปแต่ถ้าผู้บุกรุกใช้วิธีการอื่นในการยกระดับสิทธิ์ตัว Anti-Exploit ก็ไม่สามารถกันได้เราจึงมีส่วนของ เคอร์เนล โมดูล เพื่อคอยตรวจจับว่ามีการกระทำอะไรเกิดขึ้นกับ file ต่างๆ ในระบบซึ่งถ้ามีการเปลี่ยนแปลงใดๆ เกิดขึ้นกับ file ไม่ว่าจะเป็น write delete chmod chown ฯลฯ

ซึ่งเราได้ใช้เทคนิคคือ เขียน module เพื่อ Hook Systemcall เพื่อดูว่ามีอะไรเกิดขึ้นกับ file บ้างซึ่งวิธีการของ การใช้ เคอร์เนล โมดูล นั้นมี ข้อดีคือมันจะใช้ cpu time น้อยมากและทำให้ประสิทธิภาพ ของระบบเราดีขึ้นและเมื่อเรารู้แล้วว่าผู้บุกรุกได้กระทำใดๆก็ตามกับระบบของเรา เราก็ใช้ส่วนของ ความถูกต้องของไฟล์ มาเชกว่า ไฟล์ นั้นได้ถูกแก้ไขหรือไม่เมื่อไรและเวลาใดซึ่งข้อมูลต่างๆนั้นจะถูกเก็บลงบนฐานข้อมูลทั้งหมด

#### 1. ส่วนของ Anti Exploit

ส่วนนี้จะเป็นส่วนของการป้องกันการบุกรุกช่องโหว่ของซอฟต์แวร์ (Anti-Xploit) ที่ออกแบบให้เป็นระบบตรวจจับผู้บุกรุกทางคอมพิวเตอร์ โดยมีจุดประสงค์การออกแบบเพื่อป้องกันการโจมตีซึ่งหลุดรอดมาจากระบบตรวจจับผู้บุกรุกทางเครือข่าย ในส่วนแรกที่ได้ติดตั้งไว้ซึ่งสามารถป้องกันได้เฉพาะการโจมตีที่ตรงกับรูปแบบที่ได้จดจำไว้ และไม่สามารถป้องกันการบุกรุกจากผู้ใช้ในระบบที่ลือคอินเข้ามาอย่างถูกต้อง

โดยระบบป้องกันการบุกรุกช่องโหว่ของซอฟต์แวร์ที่พัฒนาจะเน้นไปที่การป้องกัน บัฟเฟอร์โอเวอร์โฟลว์ ในรูปแบบต่างๆ เนื่องจากหลักการของการทำ Exploit Code แก่นแท้ของการทำก็คือการทำ บัฟเฟอร์โอเวอร์โฟลว์ ชนิดต่างๆ

### การออกแบบส่วน Anti Exploit

หลักการที่นำมาใช้ป้องกัน บัฟเฟอร์โอเวอร์โฟลว์ คือ Linux Function Interception ( การดักฟังก์ชันใน linux ) ผ่านทางไฟล์ ld.so.preload โดยในส่วนนี้จะไปดักฟังก์ชันใน libc เท่านั้น โดยในส่วนนี้จะซึ่งเป็นการเขียนทับฟังก์ชันที่เสี่ยงต่อการถูกโจมตีด้วย บัฟเฟอร์โอเวอร์โฟลว์ โดยเมื่อมีการเรียกใช้ฟังก์ชันที่เสี่ยงผ่าน process ต่างๆ จะไปเรียกฟังก์ชันที่เราเขียนทับขึ้นมาก่อน และเมื่อตรวจสอบว่าไม่มีการล้นของ Buffer จึงค่อยไปเรียกฟังก์ชันนั้นจริงๆ แต่ถ้าพบว่าการล้นของบัฟเฟอร์ จะไม่อนุญาตให้ไปเรียกฟังก์ชันจริงๆ และจะ kill process นั้นทิ้งพร้อมทั้งบันทึกลง Log Server

### การพัฒนาส่วน Anti Exploit

- เขียนฟังก์ชันที่ต้องการดักขึ้นมาใหม่แทนฟังก์ชันที่มีความเสี่ยงได้แก่
  - Strncpy
  - memcpy
  - strcpy
  - wcsncpy
  - stpcpy
  - wcpncpy
  - strcat
  - strncat
  - wscat
  - getwd
  - realpath

ในฟังก์ชันที่เขียนมีหลักการทำงานคือ จะเปรียบเทียบบัฟเฟอร์ต้นทาง กับปลายทางว่าขนาดเกินกว่าที่บัฟเฟอร์ปลายทางจองไว้หรือไม่ ถ้าเกินก็จะไปเรียกฟังก์ชัน activation ซึ่งจะทำหน้าที่ kill process ที่มีการเรียกฟังก์ชันนั้นและ เก็บข้อมูลบันทึกลง log ในฐานข้อมูล mysql ที่เครื่อง log server แต่ถ้าไม่เกินก็จะไปเรียกฟังก์ชันเดิมจริงๆตามปกติ

## 2. ส่วน เคอร์เนลโมดูล

ส่วนของ เคอร์เนล โมดูล นั้นจะเป็น โมดูล ที่ออกแบบเพื่อเป็น ระบบตรวจจับผู้บุกรุกทางคอมพิวเตอร์ที่สามารถตรวจจับการเปลี่ยนแปลงต่างๆของทุกไฟล์ใน path สำคัญๆที่เราต้องการได้ การเปลี่ยนแปลงไฟล์ใน path ที่เราสามารถตรวจจับพบ ได้แก่

- เปลี่ยนแปลงข้อมูลในไฟล์
- เปลี่ยนแปลง modify time
- เปลี่ยนแปลงชื่อไฟล์
- เปลี่ยนแปลงสิทธิ์ของไฟล์
- เปลี่ยนแปลงผู้เป็นเจ้าของไฟล์และกลุ่ม
- เปลี่ยนแปลงที่อยู่ของไฟล์
- ลบไฟล์นั้นทิ้ง
- สร้างไฟล์ขึ้นมาใหม่
- คัดลอกไฟล์มาวางใน path ที่เราตรวจจับอยู่

โดยเมื่อ kernel module ตรวจจับพบการเปลี่ยนแปลงเหล่านี้ ก็จะเก็บบันทึกลง log ในฐานข้อมูล โดยผู้ดูแลระบบสามารถมาตรวจดูได้ผ่านทางหน้าเว็บ

### ขั้นตอนการออกแบบเคอร์เนลโมดูล

- ศึกษาการเขียน เคอร์เนล โมดูลบนลินุกซ์โดยใช้ภาษา c
- สังเกต system call ที่ถูกเรียกโดยใช้คำสั่ง strace เมื่อมีการเปลี่ยนแปลงไฟล์ในลักษณะต่างๆ  
เช่น `strace rm /home/hello.c`
- สังเกตอากิวเมนต์ของ system call เหล่านั้นเมื่อมีการเรียกในโอกาสต่างๆทั้งที่ถูกเรียกเมื่อมีการเปลี่ยนแปลงไฟล์ และถูกเรียกเมื่อไม่มีการเปลี่ยนแปลงไฟล์ เพื่อหาความแตกต่างของค่าอากิวเมนต์
- เขียน เคอร์เนล โมดูลเพื่อไป hook system call เหล่านั้น และเมื่อใดก็ตามที่ system call ถูกเรียกด้วยค่าอากิวเมนต์ซึ่งบ่งบอกว่ามีการเปลี่ยนแปลงไฟล์ ก็จะบันทึกข้อมูลลง log ในฐานข้อมูล
- สร้างไฟล์ที่สามารถใส่ path ที่ต้องการตรวจจับ และปรับปรุงให้บันทึกลงฐานข้อมูลเฉพาะการเปลี่ยนแปลงไฟล์ที่อยู่ใน path ที่ใส่ลงไปไฟล์เท่านั้น

### การพัฒนาเคอร์เนลโมดูล

มีฟังก์ชันเริ่มต้นซึ่งจะมีการทำงาน 2 ส่วนได้แก่

1. ไป hook system call ทั้งหมดที่สามารถเปลี่ยนแปลงไฟล์ในแบบที่กล่าวมาข้างต้น ได้แก่
  - `sys_call_table[__NR_open]`
  - `sys_call_table[__NR_write]`
  - `sys_call_table[__NR_unlink]`
  - `sys_call_table[__NR_chmod]`
  - `sys_call_table[__NR_chown32]`

- sys\_call\_table[\_\_NR\_utime]
- sys\_call\_table[\_\_NR\_rename]

โดยจะเปลี่ยนตำแหน่งพอยเตอร์ ของ system call เหล่านั้นให้มาชี้ที่ฟังก์ชันที่เราเขียนขึ้นมาเอง

2. สร้างไฟล์ชื่อ Watson ขึ้นมาในไดเรกทอรี proc เพื่อเป็นตัวส่งข้อมูลจาก user space เข้ามายัง เคอร์เนล โมดูล ของเรา โดยอินพุตที่ใส่เข้าไปจะเป็น path ที่เราต้องการให้ เคอร์เนล โมดูล ตรวจสอบการเปลี่ยนแปลง

มีฟังก์ชันจบซึ่งมีการทำงาน 2 ส่วนดังนี้

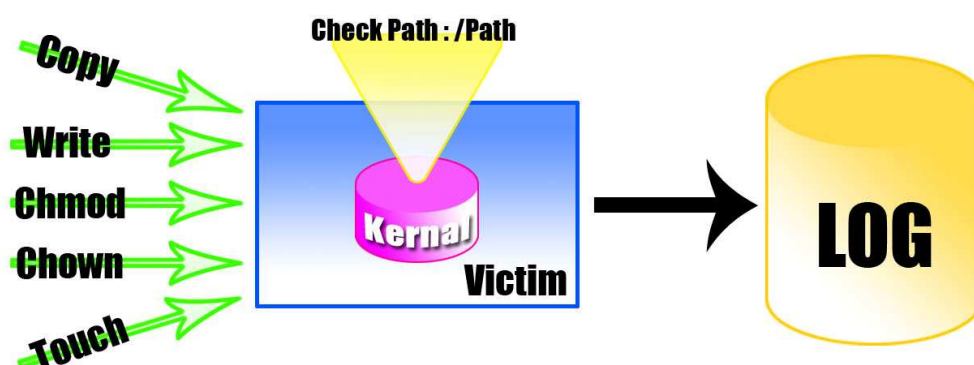
1. เปลี่ยนแปลงตำแหน่งพอยเตอร์ ของ system call ที่ hook มาตอนแรกให้ไปชี้ยัง system call จริงๆเหมือนเดิม โดยถ้าตำแหน่งพอยเตอร์ของฟังก์ชัน system call ที่เขียนขึ้นมาเองถูกเปลี่ยนก็แสดงว่ามีคนมา hook system call ของเรามากก็ จะมีการเก็บบันทึก log ลงฐานข้อมูล
2. ลบไฟล์ Watson ในไดเรกทอรี proc ที่สร้างขึ้น

เขียนฟังก์ชัน system call ที่ hook มาทั้งหมดขึ้นใหม่โดยมีหลักการทำงานดังนี้

1. นำอาทิวเม้นท์ที่ผ่านเข้ามาในฟังก์ชันซึ่งเป็นตัวบอกชื่อไฟล์ที่ถูกเปลี่ยนแปลงไปหา path เดิมๆ
2. เช็คค่า path ของไฟล์ที่มีการเปลี่ยนอยู่ในอินพุตของไดเรกทอรีที่เรา กำลังตรวจสอบอยู่หรือไม่ ถ้าไม่อยู่ก็จะไปเรียก system call ดั้งเดิมเลย ถ้าใช่ก็จะเช็คในข้อถัดไปต่อ
3. เช็คค่าอาทิวเม้นท์อื่นๆที่เกี่ยวข้องว่าเป็นค่าที่ทำให้ไฟล์นั้นเปลี่ยนแปลงหรือไม่ ถ้าไม่ก็จะไปเรียก system call ดั้งเดิมเลย แต่ถ้าพบว่ามี การเปลี่ยนแปลงไฟล์ก็จะเก็บบันทึก log ลงฐานข้อมูลก่อนจึงจะไปเรียก system call ดั้งเดิมให้

ค่าอาทิวเม้นท์ที่เกี่ยวข้องเมื่อมีการเปลี่ยนแปลงไฟล์ ได้แก่

- ใน system call sys\_open ถ้าค่าอาทิวเม้นท์ flags มีค่าเป็นเลขคู่ และค่าอาทิวเม้นท์ mode ไม่เท่ากับ 0 แสดงว่าเป็นการเปิดไฟล์นั้นๆโดยมีสิทธิ์ในการ write
- จากนั้นเมื่อมีการเรียก system call sys\_write ต่อจากการเรียก system call sys\_open แสดงว่าไฟล์ที่ถูกเปิดนั้นมีการ write เพื่อเปลี่ยนแปลงไฟล์ และถ้าเรียก system call sys\_write โดยมีค่าอาทิวเม้นท์ fd = 0 ก็จะบ่งบอกว่าเป็นการคัดลอกไฟล์อื่นมาทับไฟล์ที่เปิดโดย system call sys\_open



รูปที่ 3.3 การทำงานของ kernel Module

### 3.3.3 การกู้คืนระบบ

จะแบ่งออกเป็น 2 ส่วน คือ

#### 1. การทำ snapshot หรือ การทำ backup

ในส่วนนี้ได้เลือกใช้โปรแกรม Rsync มาเป็นเครื่องมือสำหรับการถ่ายโอนข้อมูลจากเครื่อง Response Wall มาเก็บยังเครื่อง Log Server และทำการบีบอัดด้วยโปรแกรม BZip2 ซึ่งจะทำให้ไฟล์ Snapshot มีขนาดเล็กลงเพื่อเป็นการประหยัดเนื้อที่

คุณสมบัติของโปรแกรม Rsync

- มี Difference Search Algorithm ทำให้สามารถทำ Incremental Backup ได้
- Owner group และ modify time ของไฟล์ต้นฉบับไม่เปลี่ยนแปลง
- มีการบีบอัดไฟล์ก่อนที่จะทำการส่งผ่านเครือข่าย ทำให้สามารถลดความหนาแน่นเนื่องจากปริมาณข้อมูลของเครือข่ายได้

#### 2. การ recovery

จะเป็นการป้องกันขั้นสุดท้ายคือถ้าระบบป้องกันผู้บุกรุกผ่านทางเครือข่ายและระบบตรวจจับผู้บุกรุกไม่สามารถป้องกันผู้บุกรุกได้และผู้บุกรุกได้ก่อให้เกิดความเสียหายต่อระบบซึ่งเราจะสามารถทำการกู้คืนระบบให้เหมือนเดิมก่อนที่ผู้บุกรุกจะทำให้เกิดความเสียหายได้

กระบวนการทำงานของระบบการกู้คืนระบบคือ

1. ทำการเลือกส่วนสำคัญมา backup หรือ ทำ snapshot เช่น
  - /etc เป็นที่เก็บรวบรวมไฟล์คอนฟิกต่างๆของระบบ

- /lib/module เป็นที่เก็บรวบรวมโมดูลที่จะถูกโหลดเข้าเคอร์เนลตอนเริ่มการทำงานของระบบปฏิบัติการ
  - /bin , /usr/bin เป็นที่เก็บรวบรวมโปรแกรมสำหรับใช้งานทั่วไป
  - /sbin เป็นที่เก็บรวบรวมโปรแกรมสำหรับ Super User
2. ทำการเก็บข้อมูลเพื่อเป็นการเตรียมตัว
  3. ทำการกู้คืนระบบถ้าผู้บุกรุกก่อให้เกิดความเสียหาย
- ในส่วนนี้ได้ทำการออกแบบให้ระบบสามารถทำการวิเคราะห์และค้นหาไฟล์ต้นฉบับที่เหมาะสมจาก Snapshot ที่ได้จัดเก็บไว้ และนำมาเสนอให้กับผู้ดูแลระบบเพื่อการตัดสินใจต่อไป ทั้งนี้จุดเด่นของระบบที่ได้ทำการออกแบบก็คือ การกู้คืนระบบแบบ Selective Restore โดยผู้ดูแลระบบสามารถที่จะเลือกไฟล์ที่ต้องการจะ Restore ได้ จึงไม่มีความจำเป็นที่จะต้องทำการกู้คืนระบบใหม่ทั้งหมด ทำให้สามารถประหยัดเวลา ลดปริมาณข้อมูลที่ต้องผ่านเครือข่าย และลดผลกระทบที่อาจจะเกิดขึ้นกับผู้ใช้งานระบบอื่นๆ